FLYRANK.AI · AI BACKEND ENGINEERING · PROJECT BRIEF

Containerize your stack

Run your task API against a real Postgres database in Docker — then start your whole app and its database with one command.

~4–5 h across 6 stages | Stretch +2 h | Bonus AI stage +1 h | JavaScript or Python | $0 · no credit card

COMPANION DECK	YOU WILL PRACTICE	DELIVERABLE
The developer survival kit	**Docker · Postgres · docker compose · .env secrets · parameterized queries**	**Public GitHub repo, ≥6 commits, README + DB screenshot**

How to read this document: new words are shown in **bold** the first time they appear — every one of them is explained in the Glossary at the end. Work the stages **in order**; each ends with a checkpoint that proves it works. This is the third storage swap in the same repo: memory (Task 1) → SQLite (Task 2) → containerized Postgres (this one). Continue in the same lane.

Contents

1. Goal & purpose 1
2. The big idea in 60 seconds 2
3. Tools — pick one lane 2
4. The task — six stages 3
5. Bonus stage — the AI rematch 7
6. Requirements & stretch goals 7
7. Done means 8
8. Curated resources 9
9. Glossary 11

1 · Goal & purpose

Goal: take your task **API** and run it against a real **PostgreSQL** database living in a **Docker container** — then wrap your app *and* its database so a single command, `docker compose up`, starts everything. Your endpoints behave exactly as before; your database is now a proper server, and your project runs the same way on any machine.

You've stored tasks two ways: in memory (Task 1, gone on restart) and in a **SQLite** file (Task 2, one file on disk). This brief covers the third and most professional kind of storage: a database that runs as its **own program** — PostgreSQL — the same engine behind a huge share of the world's backends, FlyRank included.

Two new habits start here, both straight from the developer survival kit. First, **Docker**: instead of installing Postgres and fighting versions, you run it as a container — a ready-made, throwaway box that behaves identically everywhere and kills "works on my machine" for good. Second, **secrets in the environment**: your database password is never hardcoded or committed — it lives in a **.env** file that Git ignores. This local stack (your app + a containerized database, started with one command) is the exact setup every later brief in this track assumes.

The reassuring part, again: **your routes barely change.** You're swapping storage for the third time — memory, then SQLite, now Postgres — and each time the API on top stays the same. The only genuinely new things are a few small infrastructure files (a Dockerfile, a compose file, a `.env`). Docker looks scary; it's mostly three commands and one YAML file.

2 · The big idea in 60 seconds

A **container** is "a machine, frozen and shippable." An **image** is the frozen recipe; a container is a running copy of it. You don't install Postgres — you run its official image and a real database appears on `localhost` in seconds. Then a second idea stacks on top: one small file describes your whole system, and one command starts it.

Word	In one line	In this project
Image	a frozen recipe of a program + everything it needs	the official `postgres` image you download
Container	a running copy of an image	your Postgres server, and later your app
Volume	disk that outlives the container	where your rows survive a restart
Docker Compose	one file that starts many containers together	`compose.yaml` = your app **and** its database

Your storage has climbed a ladder, and the API on top never noticed:

Project	Where tasks live	What runs it
Task 1	a list in memory	your program
Task 2	a `tasks.db` file	your disk (SQLite)
Task 3 (this)	rows in **Postgres**	a **container** — a real database server

Same API the whole way down. Once your app and your database both run in containers, "it works on my machine" quietly becomes "it works on every machine."

3 · Tools — pick ONE lane

Both lanes build the same stack. **Stay in the lane you've used since Task 1** — this is the same repo growing.

	JavaScript lane	Python lane
Language	Node.js (free, nodejs.org)	Python 3.10+ (free, python.org)
Your app	Reuse your Task 1/Task 2 API	Reuse your Task 1/Task 2 API
Container engine	Docker Desktop (free) or Podman	same
Database	The official `postgres` image — no install	same
Database driver	node-postgres (`pg`)	`psycopg` — or keep SQLModel, just change the URL
Config / secrets	`.env` (`node --env-file` or `dotenv`)	`.env` (`python-dotenv`)
Testing your API	curl + Hoppscotch (both free)	same
Publishing	Git + a free GitHub account	same

> **You do not install Postgres — you run its container.** Not sure about the driver? JS `pg` is the standard, no-magic choice; Py `psycopg` is the standard raw driver, but if you used **SQLModel** in Task 2 you can keep it — moving to Postgres is often just swapping the connection URL. Don't switch lanes mid-project.

4 · The task — six stages (+ one bonus)

Work stage by stage, **in order**. Each stage ends with a checkpoint you can run to prove it works. Commit to Git after every stage (that's your ≥6 commits, honestly earned). **If you only finish Stage 3, ship it anyway — a working half is worth more than a broken whole.**

Golden rule: **your routes barely change.** Keep every line that talks to the database in one small module (your **repository**); only that module and the new infrastructure files (Dockerfile, compose, `.env`) should be new. Formalizing that module into clean layers is its own later project (Layered architecture).

0 A real database in one command

~30 min

You've used a database that was a file. Now meet the kind that runs as its own program.

1. Install **Docker Desktop** (free for personal use) or Podman. Confirm it works: `docker --version`.
2. Start Postgres in one command, with a **volume** so its data persists:

```
docker run --name taskdb -e POSTGRES_PASSWORD=dev -e POSTGRES_DB=tasks \
  -p 5432:5432 -v taskdata:/var/lib/postgresql/data -d postgres
```

   Reading it: run the official `postgres` image, name it `taskdb`, set the password and a database called `tasks`, map **port** 5432 out to your machine, mount a named volume for the data, run in the background.
3. Look inside: `docker ps` (it's running), then `docker exec -it taskdb psql -U postgres -d tasks` opens **psql**, a SQL prompt inside the container. Type `\dt` (no tables yet), then `\q`.
4. Add a **.gitignore** now (before your first commit) with at least `.env` and `node_modules/` (or your Python virtualenv), and note your run command in the README.

> **CHECKPOINT** — `docker ps` shows a running `postgres` container, and `docker exec -it taskdb psql -U postgres -d tasks` opens a SQL prompt. You have a real database server on `localhost:5432`.

Commit: `Stage 0: Postgres in Docker + gitignore`

1 Connect your app: secret, driver, table

~45 min

Your app and your database are strangers. Introduce them — through a secret, not a hardcoded password.

1. Create a **.env** file (git-ignored from Stage 0) with a **connection string**, and commit a **.env.example** with the same keys and placeholder values:

```
DATABASE_URL=postgres://postgres:dev@localhost:5432/tasks
```

2. Install your **driver**: **JS** `npm install pg` · **Py** `pip install "psycopg[binary]"` (SQLModel users: keep it, point its URL at `postgresql://…`).
3. On startup, connect using `DATABASE_URL`, then **create the `tasks` table if it doesn't exist** (`id` serial **primary key**, `title` text, `done` boolean) and **seed** three example tasks *only if the table is empty* — the same first-run rule as Task 2.
4. Keep every database line in **one module** (your **repository**). If you're editing routes, stop — only this module should change.

> **CHECKPOINT** — your app connects with no error. `docker exec -it taskdb psql -U postgres -d tasks -c "\dt"` lists the `tasks` table, and a `SELECT * FROM tasks;` shows three rows. Restart the app three times — still three.

Commit: `Stage 1: connect via .env and create table`

2 Read from Postgres

~45 min

Same doors, new room behind them.

1. Point `GET /tasks` at `SELECT * FROM tasks`, and `GET /tasks/{id}` at `SELECT * FROM tasks WHERE id = $1` (**pg** uses `$1`) / `WHERE id = %s` (**psycopg** uses `%s`). These are **parameterized query** placeholders — pass the id separately, never glue it into the SQL string.
2. Unknown ids still return **404** with `{ "error": "Task not found" }`. Nothing about your API's behaviour changes — only the engine underneath.

> **CHECKPOINT** — `curl -i http://localhost:3000/tasks` → `200` + rows straight from Postgres. `curl -i http://localhost:3000/tasks/999` → `404` + the error JSON.

Commit: `Stage 2: read from Postgres`

3 Create, update, delete on Postgres

~1 h

The full cycle, now on a real server.

1. `POST /tasks` → `INSERT INTO tasks (title, done) VALUES ($1, $2) RETURNING *` (the `RETURNING` clause hands back the new row, id included). Keep the **validation**: missing/empty `title` → **400**; success → **201**.
2. `PUT /tasks/{id}` → `UPDATE tasks SET title = $1, done = $2 WHERE id = $3`. `DELETE /tasks/{id}` → `DELETE FROM tasks WHERE id = $1`. Unknown id → `404`; a successful delete returns `204` with an empty body.
3. **Stop and notice:** your CRUD API now runs on the same kind of database that powers real companies. FlyRank's stores, content, and SEO reports are Postgres rows exactly like these.

> **CHECKPOINT** — create a task, mark it done with `PUT`, then `DELETE` it — confirming each step with `GET /tasks`, all with the right status codes (`201`, `200`, `204`, `404`).

Commit: `Stage 3: full CRUD on Postgres`

4 One command for the whole stack

~45 min

Right now you start two things by hand. Make it one.

1. Write a small **Dockerfile** for your app (about six lines — the internal Docker guide has the shape for each lane).
2. Write a **compose.yaml** with two **services**: `api` (built from your Dockerfile) and `db` (the `postgres` image with a **volume**). Inside the compose network your app reaches the database by the service name `db`, *not* `localhost`:

```
services:
  api:
    build: .
    ports: ["3000:3000"]
    environment:
      DATABASE_URL: postgres://postgres:dev@db:5432/tasks
    depends_on: [db]
  db:
    image: postgres
    environment:
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: tasks
    volumes: [taskdata:/var/lib/postgresql/data]
volumes:
  taskdata:
```

3. Stop your hand-run container (`docker stop taskdb`) so the port is free, then run `docker compose up`.

> **CHECKPOINT** — from a clean state, `docker compose up` brings up app **and** database. Create a few tasks, then `docker compose down` and `up` again — your tasks are **still there**, because the volume kept them. Persistence across a full-stack restart.

Commit: `Stage 4: docker-compose the whole stack`

5 Publish to GitHub

~30 min

Your work only counts when a stranger can run it with one command.

1. Push to your **public GitHub repo** (the same repo as Task 1/2). Double-check `.env` is git-ignored and **.env.example is committed** — a leaked database password is a real incident.
2. Update your **README** with: what this is; the **one** command to run everything (`docker compose up`); which variables to set (point at `.env.example`); a table of all endpoints; one pasted `curl -i`; and a **screenshot** of your data in the database (psql `\dt` + a `SELECT`, or a free GUI like DBeaver / pgAdmin / TablePlus).
3. Confirm the round-trip: a stranger clones, copies `.env.example` to `.env`, runs `docker compose up`, and has a working API.

> **CHECKPOINT** — on a clean clone, `cp .env.example .env && docker compose up` starts the whole stack and `GET /tasks` returns the seeded tasks — in under 5 minutes, with no manual database setup.

Commit: `Stage 5: one-command stack + docs` — then push everything.

★ Make it yours — optional extras

optional · as many or as few as you like

You have a real database engine now — poke it.

None of these are required. Pick whatever sounds fun (creative alternatives welcome):

- **The mortality experiment, database edition:** run Postgres **without** a volume, create tasks, `docker rm` the container, start a fresh one — watch the data vanish. Two sentences in your README on why **volumes** exist.
- **A real health check:** `GET /health` that also runs `SELECT 1` and reports `db: "ok"`. Real companies gate deploys on exactly this.
- **Add an index** on a column you filter (e.g. `done`) and show `EXPLAIN ANALYZE` before and after on a seeded table.
- **Add Redis** to your compose file (you'll want it for later briefs) and `PING` it once on startup.
- **Slim the image** with a multi-stage Dockerfile; note the size before and after.

Commit (if you build any): `Extras: <what you added>`

5 · Bonus stage — the AI rematch

6 The AI rematch

~1 h · optional — and the most fun

You just containerized a whole stack by hand. Now hire the fastest junior developer on Earth to do it — and review their work.

You did Stages 0–5 by hand for a reason: you now know exactly what a correct containerized Postgres stack looks like. That knowledge is what turns this stage from a magic show into a **code review**.

1. **Write the prompt yourself — this is the real exercise.** Without copying from this document, write your own **prompt** asking an AI to containerize a task CRUD API onto Postgres. From memory, specify: your lane and driver, the `tasks` table and seed-once rule, the five endpoints keeping identical behaviour, **parameterized queries**, the password coming from **.env** (never hardcoded), a **volume** for persistence, and one-command startup with docker compose.
2. **Generate in quarantine.** Put the AI's version in a separate folder (`ai-version/`) or branch. Your Stages 0–5 stack stays untouched — that hand-built version is your deliverable.
3. **Run it.** Does `docker compose up` work first try? Create tasks, then `down` and `up` — did the data survive, or did it forget the volume? Did it hardcode the password or read it from `.env` ?
4. **Diff it.** Compare the AI's compose/Dockerfile/storage with yours (`git diff --no-index`). Then answer in an **"AI vs me"** README section: What did it do *better* (a healthcheck, `depends_on` conditions, a slimmer image)? What did it get *wrong* or ignore (a missing volume, a hardcoded secret, `localhost` instead of the service name)? What did *your prompt* forget to specify?
5. **One rematch.** Improve your prompt, regenerate, and note in one sentence what changed.

> The lesson hiding in this stage: an AI's output is exactly as good as your specification — and you could only judge it because you built the thing yourself first. **Both halves of that sentence are your career from now on.**

> **CHECKPOINT** — your README has an "AI vs me" section containing your full prompt and at least three concrete differences you found.

Commit: `Stage 6: AI vs me` (AI code stays in its own folder/branch).

6 · Requirements

Done = every box ticked. Each one is checkable in under a minute.

- [ ] **Postgres runs in a container**, and the whole stack starts with a single `docker compose up` .
- [ ] The app connects using a **connection string from** `.env` (git-ignored; `.env.example` committed) — **no hardcoded credentials** anywhere.
- [ ] The **tasks table is created automatically** if missing, and three example tasks are **seeded only on the first run**.
- [ ] All five CRUD endpoints — `GET /tasks` , `GET /tasks/:id` , `POST /tasks` , `PUT /tasks/:id` , `DELETE /tasks/:id` — work against Postgres with the **same shapes** as Tasks 1 and 2, using **parameterized queries**.
- [ ] Correct status codes: `200` / `201` / `204` success, `400` invalid body, `404` unknown id — each error with a JSON `error` message.
- [ ] **Data persists** across a full `docker compose down` then `up` (a **volume** keeps it).
- [ ] Public **GitHub repo** updated: README with the one-command run, `.env.example` , an endpoint table, one `curl -i` output, and a **screenshot of the data in the database**.

Stretch (optional)

- [ ] **Prove the swap:** keep your Task 1/2 endpoint tests (or the same curl calls) and show they pass unchanged against Postgres — then explain why identical behaviour across three storage engines proves storage is "just an implementation detail" (and why a layered architecture formalizes exactly this).
- [ ] Add a **health check** that pings the database, and write one line on what a load balancer does with it.
- [ ] Ship a **multi-stage Dockerfile** and report the image size before and after.
- [ ] Stage 6 — the AI rematch (see above): prompt an AI to containerize the same stack, run it, diff it, write your "AI vs me" section.

7 · Done means

- On a clean clone, `cp .env.example .env && docker compose up` brings up the whole stack, the full CRUD cycle works via `curl -i` (right status codes visible), and the same rows are visible in the database.
- **Persistence is proven:** create tasks, `docker compose down` then `up`, and they're still there.
- Your repo is public, `.env` appears nowhere in git history, and `git log` shows one honest commit per stage.

8 · Curated resources

Don't read everything. Each section says *when* you need it. All resources are free, no credit card. Rows marked JS are JavaScript-lane, Py are Python-lane.

● **Start here** (everyone) · ● **When you're comfortable** · ● **Optional deep dive**

§1 · Containers & Docker — the mental model

	Resource	Format	Why it's useful
●	Internal — Docker survival guide	Internal guide	Image vs container, the one-line database, your first Dockerfile, and the exact compose file this project needs. Start here.
●	Docker — Get started (official)	Official, ~30 min	The guided intro to what Docker is and the core commands. Parts 1–3 are plenty for Stage 0.
●	Docker for Beginners — TechWorld with Nana	Video, ~3 h	The classic animated walkthrough. Watch through the Compose section; skip the rest for now.
●	Play with Docker	Browser playground	A free in-browser Docker environment if a local install fights you.

§2 · Postgres in a container

	Resource	Format	Why it's useful
●	postgres — official image (Docker Hub)	Image docs	The canonical reference: the `POSTGRES_PASSWORD` / `POSTGRES_DB` variables, the port, and where data lives (the path you mount a volume onto).
●	Use the Postgres Docker Official Image (Docker blog)	Article, ~15 min	A friendly official walkthrough of running Postgres in a container, connecting, and persisting data with a volume — exactly Stage 0.
●	Internal — SQL basics guide	Internal guide	The CREATE TABLE / INSERT / SELECT for the `tasks` table and the seed, plus the common traps to watch out for.

§3 · Connect your app — the database driver

	Resource	Format	Why it's useful
●	JS node-postgres (`pg`) — Queries	Official docs	How to connect and run **parameterized** queries with `$1` placeholders — the calls behind all of CRUD. See also the npm page.
●	Py psycopg 3 — Basic usage	Official docs	Connect, get a cursor, execute, fetch rows. Pair with Passing parameters for the `%s` placeholders that keep queries safe.
●	Py SQLModel — with PostgreSQL	Official docs	If you used SQLModel in Task 2, you often only change the connection URL to move to Postgres. The ORM path for the Python lane.

§4 · One command for the whole stack — docker compose

	Resource	Format	Why it's useful

●	Docker Compose — Get started (official)	Official tutorial	Builds a small multi-container app with a compose file end to end — the shape Stage 4 asks for.
●	Postgres with Docker & docker-compose — geshan	Article, ~9 min	A tight, verified walkthrough from `docker run` to a compose file **with a volume**, then wiring it into an app. The closest worked example to Stages 0–4.
●	JS Docker Compose Node.js & Postgres — BezKoder	Article, ~30 min	A full JS build-along: Express + `pg` + Postgres in compose, with a matching GitHub repo to compare against.
●	Py Dockerizing FastAPI with Postgres — TestDriven.io	Article, ~40 min	A thorough Python build-along: FastAPI + Postgres in Docker Compose. Read the first half (skip the Traefik/production parts).

§5 · Secrets & config — the .env file

	Resource	Format	Why it's useful
●	Internal — Env vars & secrets guide	Internal guide	The `.env` workflow, the `.env.example` habit, and the three unbreakable rules — including what to do the minute a key leaks. Stage 1 and Stage 5.
●	The Twelve-Factor App — Config	Article, ~3 min	The "why" in three minutes: code identical everywhere, config in the environment. The principle behind keeping secrets out of Git.

§6 · Git & GitHub — publish your work (Stage 5)

	Resource	Format	Why it's useful
●	Internal — Git & GitHub guide	Internal guide	The daily loop (add → commit → push), plus the `.gitignore` rules that keep `.env` and `node_modules/` out of your repo.
●	GitHub Docs — Hello World	Official, ~15 min	If your repo setup is hazy, the fastest refresher on repos, commits, and pushing to GitHub.

Per-stage critical resources

Stage	Keep these open
0 · database in Docker	Internal Docker guide + the `postgres` image page (§1–2)
1–3 · connect & CRUD	Your lane's driver docs (§3) + internal Env vars guide (§5)
4 · compose the stack	Docker Compose quickstart + the geshan walkthrough (§4)
5 · publish	Internal Git & GitHub guide (§6)

9 · Glossary

Plain-language definitions of every **bold** word above. No definition depends on another — read them in any order.

Word	What it means
API	The set of doors your program offers so other programs can talk to it — here, the five task endpoints. It stayed the same across all three storage swaps.
Docker	A tool that packages a program with everything it needs and runs it in an isolated box (a container) that behaves the same on every machine.
Image	A frozen, complete recipe of a program and its dependencies. You download the official `postgres` image; nothing is installed onto your own system.
Container	A running copy of an image — like an object created from a class. You can start, stop, and delete containers without touching the image.
Docker Hub	The public library of ready-made images. The official `postgres` image you run comes from there.
PostgreSQL / Postgres	A powerful, free, open-source database that runs as its own server program. The engine behind a large share of real backends, FlyRank included.
psql	The command-line SQL prompt for Postgres. `docker exec -it taskdb psql …` opens one inside your database container.
Port	A numbered doorway on a machine. `-p 5432:5432` maps your computer's port 5432 to the container's, so your app can reach the database.
Volume	Storage that lives outside a container and outlives it. Without one, a container's data dies when the container is removed; with one, your rows survive restarts.
Environment variable	A named value the operating system hands your program at startup — used to keep configuration and secrets out of your code.
.env / .env.example	`.env` is a git-ignored file holding your real secrets (like the database password); `.env.example` is a committed copy with placeholder values so others know which keys to set.
Connection string	A single line telling your app how to reach the database: `postgres://user:password@host:port/dbname`. Yours lives in `.env`.
Driver	The library your language uses to talk to Postgres — `pg` for Node, `psycopg` for Python. It sends your SQL and returns rows.
Repository	The one module in your code that talks to the database, kept separate from your routes so swapping storage touches only this file.
Dockerfile	The build script for your app's own image: which base to start from, how to install dependencies, and what command runs the app.
Docker Compose	A tool that reads one file (`compose.yaml`) describing several containers and starts them together with `docker compose up`.
Service	One container defined in a compose file. Yours has two services, `api` and `db`, and they reach each other by service name.
Parameterized query	A query where you leave a placeholder (`$1` in `pg`, `%s` in `psycopg`) for user input and pass the value separately, instead of pasting it into the SQL text. It keeps user data from breaking or attacking your database.

Seed	To insert starting example data the first time an app runs — here, the three example tasks, inserted only when the table is empty.
Persistence	The quality of data staying available after a program (or container) stops and starts again. A volume is what gives your Postgres data persistence.
Primary key	A column whose value uniquely identifies each row. `id` is the primary key of `tasks`; Postgres fills it in for you.
Table	A collection of related data arranged in rows and columns, like one sheet in a spreadsheet. You have one table, `tasks`.
SQL	Structured Query Language — the language for creating, reading, updating, and deleting data in a database. `SELECT * FROM tasks` is SQL.
CRUD	Create, Read, Update, Delete — the four basic things an app does with its data, and the four operations you moved onto Postgres.
Validation / validate	Checking incoming data before trusting it (is `title` present and non-empty?) and rejecting bad input with a `400`. Carried over unchanged from Tasks 1 and 2.
Status code	The 3-digit number in every response saying how it went: `200` OK, `201` Created, `204` No Content, `400` Bad Request, `404` Not Found.
.gitignore	A file listing paths Git should never track — like `.env` and `node_modules/`. Create it before your first commit.
Prompt	The instructions you give an AI assistant. In Stage 6 your prompt is a mini-specification of the whole stack — the more precisely it names the table, secrets, volume, and endpoints, the closer the AI's output lands to yours.
Diff	A line-by-line comparison of two versions of code showing what was added, removed, or changed. `git diff` produces one; reading diffs is how professionals review each other's work.
Git / GitHub / repo / commit	Git tracks versions of your code in a **repo**sitory; a **commit** is one saved step with a message; GitHub hosts the repo online so others can clone and run it.
README	The front page of a repo: what the project is, how to run it, and how it works.